MEDICARE PORTAL

EXHAUSTIVE & DETAILED TECHNICAL DOCUMENTATION

MERN Stack Infrastructure, Database Models, REST API Routes, Controllers, Scraping Workflows, Session Security, and Launch Scripts.

Generated on: 6/2/2026, 12:02:10 AM

Core Environment: Node.js Express REST API, React SPAs, MongoDB Atlas
Verified Standards: BM&DC Automated scraper, CAPTCHA Solver, OTP Auth

TABLE OF CONTENTS

This document outlines the exhaustive specifications of the Medicare healthcare portal:

- **1. Project Overview & Functional Modules:** Detailed purpose, objectives, and roles.
- **2. Comprehensive File and Directory Map:** Folder and file breakdown of backend, frontend, and admin.
- **3. Technology Stack & Package Mapping:** Complete programming languages and backend/frontend npm dependencies.
- **4. Complete Database Models & Schema Specifications:** Structural fields, types, indexes, and relations for all 14 Mongoose models.
- **5. Detailed API Endpoint Registry:** Exhaustive route map with methods, guards, and details.
- **6. Detailed Controller Functions Map:** Listing and functions in the 12 controller modules.
- **7. Session Security & Mutual Exclusion Architecture:** Token crosstalk preventions, React storage synchronization.
- **8. Automatic BM&DC Verification Scraper Design:** Captcha solver, JIMP image transformations, Cheerio parser.
- **9. Launch Scripts & Infrastructure Spec:** Complete launch launcher script, git sync script, render.yaml configuration.

1. PROJECT OVERVIEW & FUNCTIONAL MODULES

Medicare is a healthcare platform built using the MERN stack. It handles patients seeking consultations, doctors providing services, and admins monitoring clinical audits.

Patient Portal Roles

- **Doctor Selection:** Filters doctors by specialization, bio, ratings, location, and consultant fees.
- **Booking & Payments:** Creates offline or video slots and handles sandbox checks via Stripe and Aamarpay.
- **Health Logs:** Records blood pressure, blood glucose, temperature, weight, and keeps private medical diaries.
- **Medical Files:** Uploads NID scans, clinical lab reports, and files (PDF/images) directly to database slots.

Doctor Portal Roles

- **Schedule Manager:** Maintains active date slots, blocks individual time slots, and configures blackout ranges.
- **Digital Prescriptions:** Generates prescriptions outlining symptoms, diagnoses, medicine advice, dosage, and duration.
- **Consultation Chats:** Exchanges text messages in real-time with patients booked for consultations.

Admin Portal Roles

- **Doctor Audits:** Reviews doctor applications and overrides verification statuses to Verified or Rejected.
- **System Audits:** Views paginated system logs detailing admin logins and security alerts (unrecognized IP).

2. COMPREHENSIVE FILE AND DIRECTORY MAP

The project contains three workspaces (backend, frontend, admin) and root orchestration scripts.

Root files

- **LAUNCH_MEDICARE.bat:** Automates check installs, builds, and launches dev servers for all three components.
- **SYNC_TO_GITHUB.bat:** Initializes git, adds remotes, commits local changes, and forces main push.
- **render.yaml:** Configures infrastructure-as-code mappings for backend API, frontend static web, and admin static web.

Backend Workspace Folder Structure

- **backend/controllers/:** Contains 12 controller files implementing MERN service actions.
- **backend/routes/:** Contains 13 routers exposing API paths to frontend clients.
- **backend/models/:** Contains 14 database models defining Mongoose MongoDB collections.
- **backend/middlewares/:** Implements guards: firebaseAuth.js, doctorAuth.js, adminAuth.js.
- **backend/utils/:** Integrates services: bmdcScraper.js, email.js (Nodemailer), cloudinary.js (uploads).

3. TECHNOLOGY STACK & PACKAGE MAPPING

The tech stack leverages JavaScript ES6+ modules in Node.js backend and React SPAs built with Vite.

Backend Dependencies

- **express (v5.2.1) & cors (v2.8.5):** REST routing frameworks.
- **mongoose (v9.0.1) & mongodb (v7.2.0):** NoSQL database client.
- **jsonwebtoken (v9.0.3):** Access signatures.
- **bcryptjs (v3.0.3):** Hashes credentials.
- **tesseract.js (v7.0.0):** OCR CAPTCHA solver.
- **jimp (v1.6.1):** Image filters.
- **cheerio (v1.2.0):** HTML scraper.
- **pdfkit (v0.18.0):** Digital PDF prescriptions generator.
- **nodemailer (v8.0.10):** Transactional SMTP OTP emails.
- **cloudinary (v2.8.0) & multer (v2.0.2):** File upload streams.
- **stripe (v20.0.0) & axios (v1.16.1):** Payments and client requests.

Frontend Dependencies

- **React (v19.1) & react-dom (v19.1):** UI layer.
- **vite (v7.1) & @tailwindcss/vite (v4.1.17):** Vite build tool and TailwindCSS.
- **react-router-dom (v7.9):** SPA routes.
- **firebase (v12.13.0):** Authentication client.

4. DATABASE MODELS & SCHEMA SPECIFICATIONS

MongoDB collections are specified below detailing every single Mongoose field, index, and relational link.

1. Doctor Model (Doctor.js)

- **email (String, unique, index):** Housed lowercase, validated practitioner email.
- **password (String, select: false):** Securely hashed password string.
- **bmdcNumber (String, unique):** Registration number for council verification.
- **verificationStatus (String):** Enum: 'Unverified', 'Pending', 'Verified', 'Rejected'. Default: 'Unverified'.
- **isVerified (Boolean, default: false):** Verification status flag.
- **schedule (Map of arrays):** Time slots by date (e.g. {'YYYY-MM-DD': ['Time']}).
- **blockedSlots (Array):** Unavailable slots: {date, slot}.
- **blackoutPeriods (Array):** Ranges: {startDate, endDate, reason}.
- **pricingTiers:** Tier fees: {video (default 500), offline (default 400)}.

2. Patient Model (PatientProfile.js)

- **clerkUserId (String, unique, index):** External identifier.
- **email (String, unique, sparse, index):** Lowercase email.
- **phone (String, unique, sparse, index):** Contact phone.
- **otp / otpExpires (String / Date):** Activation OTP and 10-min expiry.
- **nid (String):** National Identity Card number.
- **nidImageUrl / nidImagePublicId:** NID image cloud properties.
- **isVerified / verificationStatus:** Identity status properties.
- **medicalHistory (Array):** Subdocs: {condition, date, notes, fileUrl, filePublicId}.
- **bookmarkedArticles (Array of ObjectIds):** Refs to Article collection.
- **latestSymptomCheck:** Checks: {symptoms, recommendedSpecialty, checkedAt}.

3. Appointment Model (Appointment.js)

- **doctorId / userId (ObjectId):** Foreign references to Doctor and PatientProfile.
- **date / time (String):** Appointment slot.
- **fees (Number):** Amount charged.
- **paymentStatus (String):** Enum: 'Unpaid', 'Paid', 'Refunded'.
- **status (String):** Enum: 'Pending', 'Accepted', 'Completed', 'Canceled', 'Rescheduled'.
- **paymentGateway / transactionId:** Gateway details (stripe/aamarpay).

4. Health Log Model (HealthLog.js)

- **userId (ObjectId):** Patient profile ref.
- **bloodPressure (Object):** BP: {systolic (Number), diastolic (Number)}.
- **bloodSugar (Number) / temperature (Number):** Sugar level (mg/dL) / temp (Fahrenheit).

- **weight (Number) / date (String):** Weight (kg) and recording date.

5. Journal Model (Journal.js)

- **userId (ObjectId):** Patient profile ref.
- **title / date (String):** Journal entry properties.
- **entries (Array):** Subdocs: {entryId, entryText, mood, createdAt}.
- **cheers (Array of ObjectIds):** Refs to patients who cheered the public entry.

6. Message Model (Message.js)

- **appointmentId (ObjectId):** Reference to Appointment.
- **senderId / receiverId (String):** Sender and receiver user IDs.
- **message (String) / timestamp (Date):** Message text and delivery time.
- **isRead (Boolean):** Read status.

7. Prescription Model (Prescription.js)

- **doctorId / patientId / appointmentId (ObjectId):** References to Doctor, Patient, Appointment.
- **date (String) / diagnoses (Array of Strings):** Date and diagnoses list.
- **medicines (Array):** Subdocs: {name, dosage, frequency, duration}.
- **advice (String):** General medical instructions.

8. Audit Log Model (AuditLog.js)

- **adminEmail / adminRole:** Email and role of admin performing action.
- **action / details / ipAddress:** Logged action name, details, client IP.
- **timestamp (Date):** Time of audit event.

9. Other Models (Admin, Service, serviceAppointment, Article, Post)

- **Admin.js:** Stores admin credential, role, knownIps array, and lastLoginIp.
- **Service.js:** Clinical services: {name, description, price, duration, imageUrl, category}.
- **serviceAppointment.js:** Clinical booking: {serviceId, userId, date, time, status, amount, transactionId}.
- **Article.js / Post.js:** Health articles / Social QA posts with like/comment schemas.

5. DETAILED API ENDPOINT REGISTRY

The following table represents every single REST API route exposed by the backend system:

Admin Routes (/api/admin)

- **POST /login:** Logs in admins. Triggers IP whitelisting / security alert alerts on unrecognized IPs.
- **GET /audit-logs:** Lists paginated system audit entries. Guard: adminAuth (super-admin only).
- **GET /me:** Returns details of the authenticated administrator. Guard: adminAuth.

Doctor Routes (/api/doctor)

- **POST /signup:** Processes doctor signup. Calls automatic scraper. Returns success status.
- **POST /login:** Authenticates credentials, updates passwords to hashed formats on fallback match, returns JWT.
- **GET /:** Returns a list of doctors with completed appointments, earnings, and availability.
- **GET /:id:** Retrieves details of a doctor profile by ID.
- **PUT /:id:** Updates doctor schedule, blackout ranges, pricing tiers. Guard: doctorAuth.
- **PUT /:id/certificate:** Uploads certificate scan to Cloudinary. Guard: doctorAuth.
- **POST /:id/verify-certificate-online:** Triggers scraper credential search on bmdc.org.bd. Guard: doctorAuth.
- **POST /:id/approve-verification:** Overrides doctor verification status (Verified/Rejected). Guard: adminAuth.
- **POST /:id/toggle-availability:** Toggles availability between Available and Unavailable. Guard: doctorAuth.
- **DELETE /:id:** Deletes doctor account and removes avatar image from Cloudinary. Guard: adminAuth.

Patient Routes (/api/patient)

- **POST /signup:** Saves unverified patient profile, generates 6-digit OTP code, and emails code via SMTP.
- **POST /verify-otp:** Validates sign-up OTP and activates patient profile.
- **POST /login:** Logs in patient and returns profile JWT.
- **POST /forgot-password:** Generates resetOtp code (15-min expiry) and emails it to the patient.
- **POST /reset-password:** Validates resetOtp and saves new hashed password.
- **GET /profile:** Retrieves profile, medical files, bookmarks. Guard: requireFirebaseAuth.
- **PUT /profile:** Updates details and uploads NID image. Guard: requireFirebaseAuth.
- **PUT /profile/medical-history:** Uploads report file (PDF/Image) to medical history. Guard: requireFirebaseAuth.
- **DELETE /profile/medical-history/:itemId:** Removes medical history file. Guard: requireFirebaseAuth.
- **POST /profile/bookmarks/:articleId:** Toggles bookmark on a health article. Guard: requireFirebaseAuth.
- **GET /profile/bookmarks:** Lists bookmarked articles. Guard: requireFirebaseAuth.
- **PUT /profile/symptom-check:** Updates latest check results. Guard: requireFirebaseAuth.
- **GET /profile/:clerkUserId:** Retrieves patient medical history for doctor view. Guard: doctorAuth.
- **GET /profiles:** Retrieves all patient profiles. Guard: adminAuth.
- **POST /profiles/:clerkUserId/verify:** Overrides patient identity verification to Verified. Guard: adminAuth.

Appointment Routes (/api/appointment)

- **POST /:** Books appointment, checks for schedule blocks, calculates fees, returns gateway checkout.
- **POST /aamarpay/callback:** Processes Aamarpay payment webhooks, sets paymentStatus='Paid' on success.
- **GET /confirm:** Validates Stripe redirect session and confirms payments.
- **GET /stats/summary:** Returns clinic metrics (earnings, completion ratios, counts).
- **GET /me:** Lists appointments for the authenticated patient. Guard: requireFirebaseAuth.
- **GET /doctor/:doctorId:** Lists appointments booked under a doctor.
- **POST /:id/cancel:** Cancels appointment and clears doctor schedules.
- **GET /patients/count:** Returns count of patients.
- **GET /:appointmentId/intake-summary:** Retrieves intake metrics. Guard: hybridAuth (patient or doctor).
- **PUT /:id/check-in:** Self-checks in patient, updates queue state to CheckIn. Guard: requireFirebaseAuth.
- **PUT /:id/queue-state:** Transition queue status (CheckIn -> InConsultation -> Done). Guard: doctorAuth.
- **GET /queue-board/:doctorId:** Gets live list of today's checked-in queue.

Prescription Routes (/api/prescription)

- **POST /:** Creates or updates patient digital prescription. Guard: doctorAuth.
- **GET /patient:** Lists prescriptions for authenticated patient. Guard: requireFirebaseAuth.
- **GET /history/patient/:patientId:** Lists patient prescription history. Guard: doctorAuth.
- **GET /appointment/:appointmentId:** Gets prescription for appointment. Guard: readPrescriptionAuth.

Journal & Log Routes (/api/journal & /api/health-log)

- **GET /api/journal/my-journal:** Gets private entries. Guard: requireFirebaseAuth.
- **POST /api/journal/:** Creates or updates journal. Guard: requireFirebaseAuth.
- **POST /api/journal/entries:** Adds entry. Guard: requireFirebaseAuth.
- **POST /api/journal/:journalId/entries/:entryId/cheer:** Cheers a public entry. Guard: journalAuth.
- **GET /api/health-log/:** Gets logs. Guard: requireFirebaseAuth.
- **POST /api/health-log/:** Logs BP, blood glucose, temperature, weight. Guard: requireFirebaseAuth.

6. DETAILED CONTROLLER FUNCTIONS MAP

Medicare utilizes modular backend controllers. Here is an index of all exported controller functions:

appointmentController.js

- **createAppointment:** Extracts doctorId, date, time. Checks doctor schedules and blackout periods. Sets status='Pending'.
- **handleAamarpayCallback:** Receives payment IPN status. Saves paymentStatus='Paid' and transactionId to appointment.
- **updateQueueState:** Transitions live queue state (Pending -> CheckIn -> InConsultation -> Completed).
- **getQueueBoard:** Queries today's appointments for doctor, sorted by check-in timestamp.

doctorController.js

- **signupDoctor:** Performs doctor registration, invokes BM&DC scraper, falls back to manual review on timeout.
- **doctorLogin:** Validates password hashes. Upgrades old plain-text database accounts on matching passwords.
- **cleanupDoctorPastSlots:** Iterates schedules, flags slots older than current server date, deletes past items.

patientProfileController.js

- **patientSignup:** Creates unverified document, generates random 6-digit OTP, dispatches email alert.
- **patientVerifyOtp:** Compares client OTP with database value, confirms expiry, activates profile.
- **updateProfile:** Handles file upload streams for avatars and NID cards to Cloudinary folder destinations.

prescriptionController.js

- **createOrUpdatePrescription:** Saves symptoms, medicine list (name, dosage, frequency, duration), and digital signature.

messageController.js

- **sendMessage:** Stores sender, receiver, chat text, and timestamp. Logs chat interaction.

7. SESSION SECURITY & MUTUAL EXCLUSION ARCHITECTURE

To enforce patient privacy and doctor confidentiality, the React client utilizes storage event observers.

Double-Token Isolation

Doctors and patients have distinct security contexts. Doctors authenticate via a custom server JWT stored in local storage as doctorToken_v1. Patients authenticate via Firebase Client Tokens saved as patientToken_v1.

Conflict Resolution Flow in Login.jsx

The frontend login component executes explicit separation routines:

- **Doctor Sign-in/Signup:** On doctor login, the code purges patientToken_v1 from localStorage and calls the AuthContext's logout() to destroy Firebase patient states.
- **Patient Sign-in/OTP:** On patient verification, the code purges doctorToken_v1 from localStorage and dispatches a storage synchronization event to reset header contexts.

8. AUTOMATIC BM&DC VERIFICATION SCRAPER DESIGN

The signup flow intercepts doctor registrations to verify credentials against the official Bangladesh Medical and Dental Council registry.

The Scraping & CAPTCHA Solver Pipeline

- **1. Cookie Extraction:** Hits verify.bmdc.org.bd via Axios. Cheerio loads the page, extracting CSRF tokens and cookies.
- **2. CAPTCHA Retrieval:** Downloads CAPTCHA image using session cookies to ensure registry session continuity.
- **3. Jimp Image Filter Pipeline:** Converts captcha buffer to grayscale. Resizes image 300% (to 300x90). Adjusts contrast by +0.8. Applies binarization scan: pixels below 130 value become black (0), others become white (255) to isolate character noise.
- **4. Tesseract.js OCR:** Configures Tesseract worker with tessedit_char_whitelist and PSM 8. Solves the 4-character code.
- **5. Search POST Request:** Submits request to verify.bmdc.org.bd/regfind. Cheerio parses results for name and active status.
- **6. Name Fuzzy Validation:** Strips prefixes ('Dr.'), removes non-alphabetic chars, and performs substring matching.
- **7. Retry and Fallback:** Retries up to 6 times. If external portal is down, registers profile in a 'Pending' review state.

9. LAUNCH SCRIPTS & INFRASTRUCTURE SPEC

Production deployments and local development launchers are fully automated via orchestrator files.

Smart Dev Launcher (LAUNCH_MEDICARE.bat)

```
@echo off
echo ========================================
echo   MEDICARE PROJECT - SMART LAUNCHER
echo ========================================
echo.
echo [*] Starting Backend...
start "MEDICARE BACKEND" cmd /k "cd backend && if not exist node_modules (npm install) && npm run dev"
echo [*] Starting Frontend...
start "MEDICARE FRONTEND" cmd /k "cd frontend && if not exist node_modules (npm install) && npm run dev"
echo [*] Starting Admin Panel...
start "MEDICARE ADMIN" cmd /k "cd admin && if not exist node_modules (npm install) && npm run dev"
echo.
echo ========================================
echo '  Launching initiated! http://localhost:5173
echo ========================================
```

Production Orchestrator Configuration (render.yaml)

```
services:
  - type: web
    name: medicare-backend
    env: node
    plan: free
    buildCommand: npm install
    startCommand: node index.js
    envVars:
      - key: NODE_ENV
        value: production
      - key: PORT
        value: 10000
      - fromContext: MONGODB_URI
      - fromContext: AAMARPAY_STORE_ID
      - fromContext: AAMARPAY_SIGNATURE_KEY
  - type: static
    name: medicare-frontend
    env: static
    buildCommand: npm install && npm run build
    publishDir: dist
    envVars:
      - key: VITE_API_URL
        value: https://medicare-backend-6eww.onrender.com
```